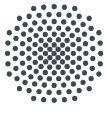


University of Stuttgart
Germany

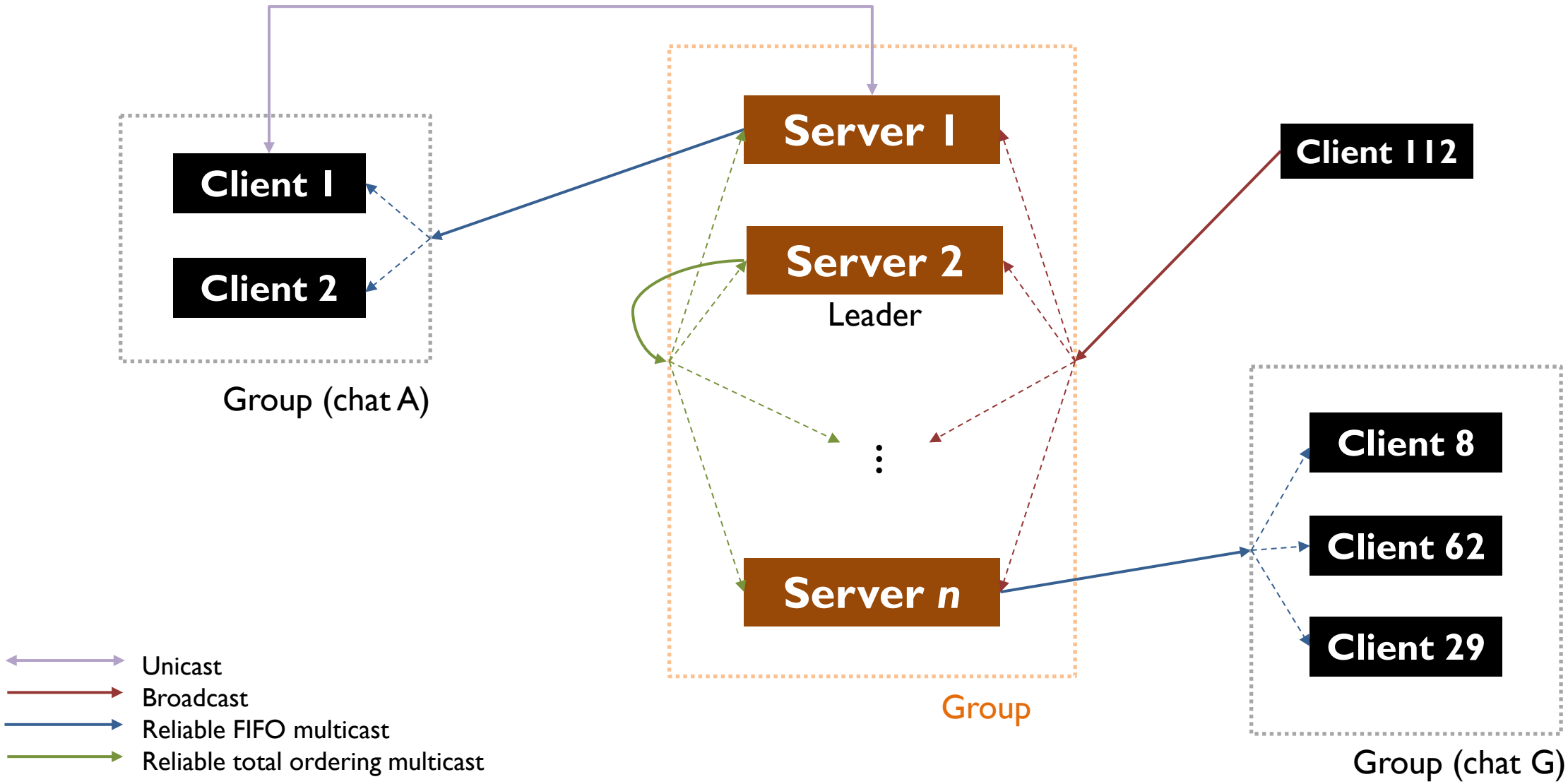
Dynamic discovery of hosts

Discovery objectives, custom broadcast-based dynamic discovery

Ilche Georgievski
ilche.georgievski@iaas.uni-stuttgart.de

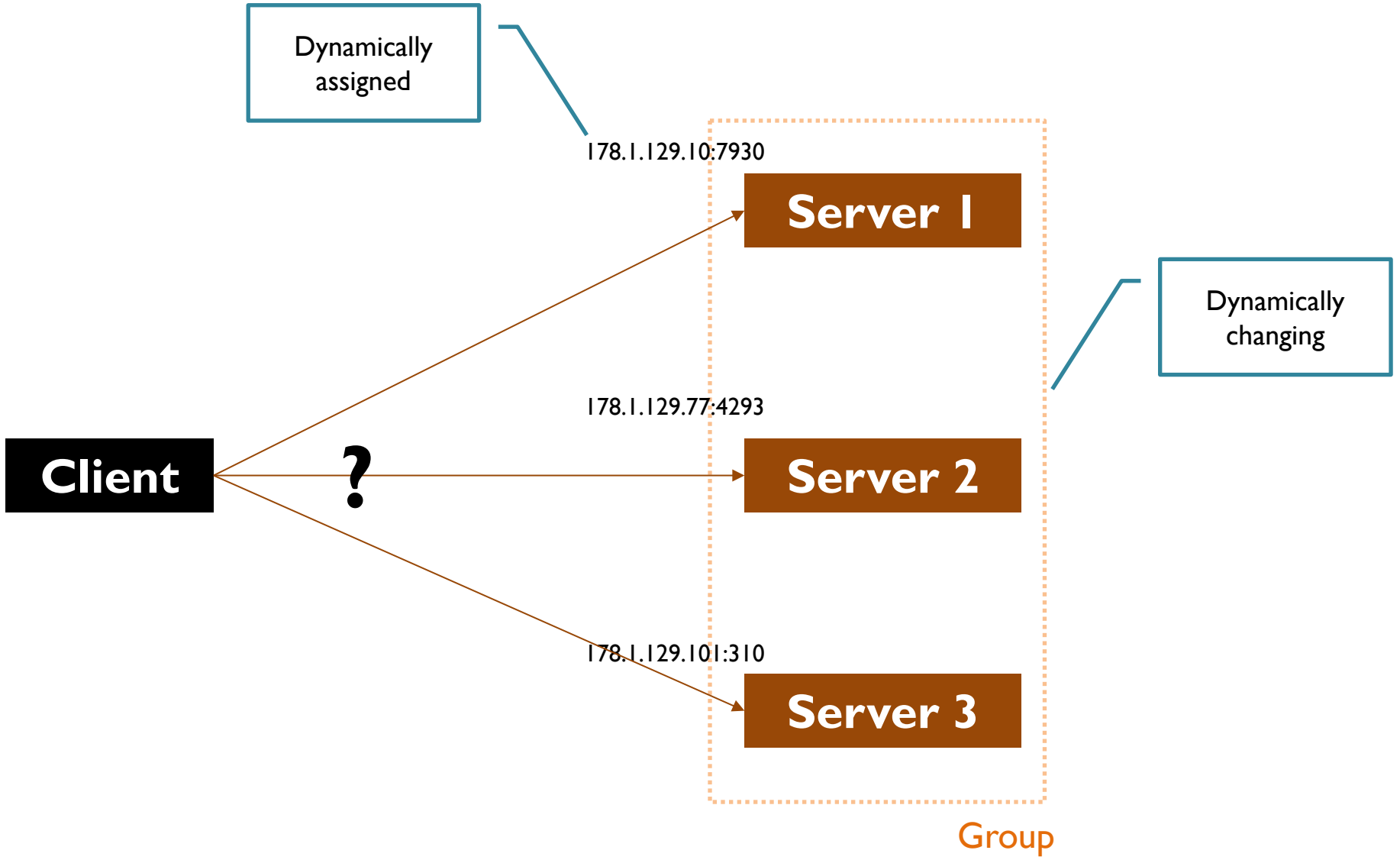
Identification of communication participants is a necessary precondition to any communication

Chat application



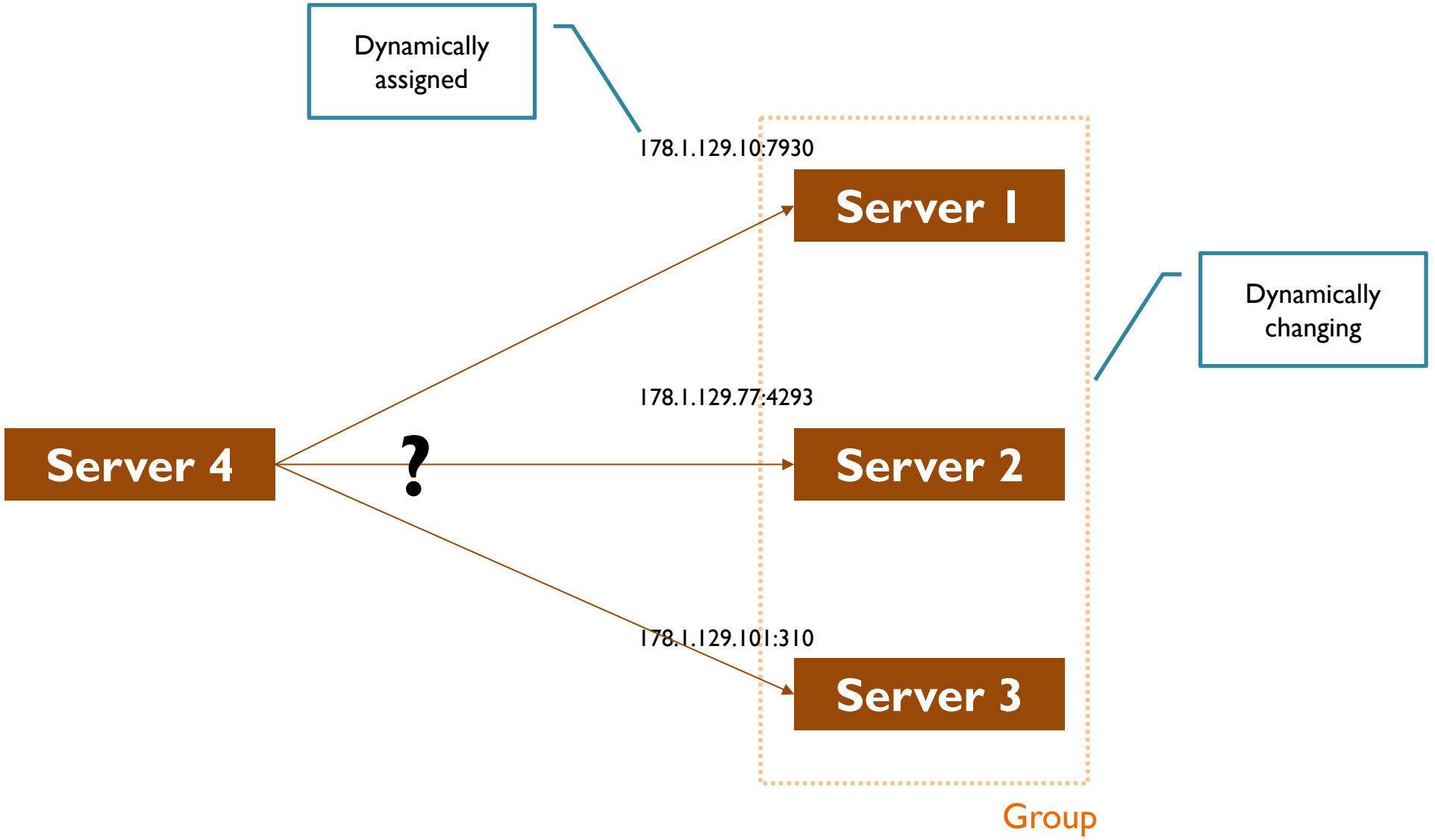
Problem 1

How to find which server to connect to?



Problem 2

How to identify participants of a group and become a participant?



Discovery's objectives

- Reachability objective
  - Enable a new (joining) node to find a participant that fulfills a specific role (e.g., leader, chat moderator)
- System availability objective
  - Enable a new node to find and join an appropriate group (e.g., replicas, peers)

- Discovery allows a new node to learn **which participants exist** and **how to reach them**

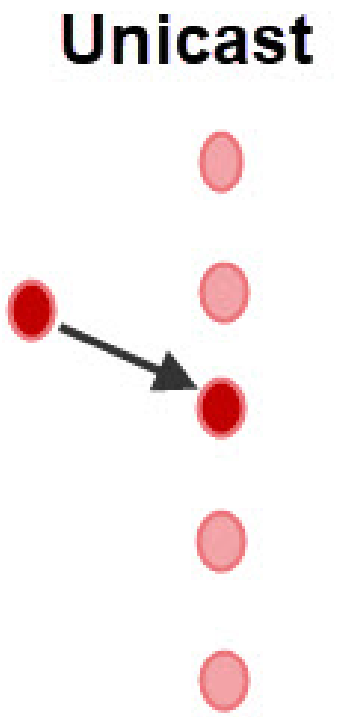
Problems with static/centralised approaches

- IP addresses of participants may not be known to programmers
- IP addresses of participants change → static configuration is not practical
- Single point of failure
- Bottlenecks and limited scalability
- High latency under load
- Poor fit for dynamic distributed environments
- Configuration complexity

Dynamic discovery

How can a new node find existing participants when it has no prior knowledge about them?

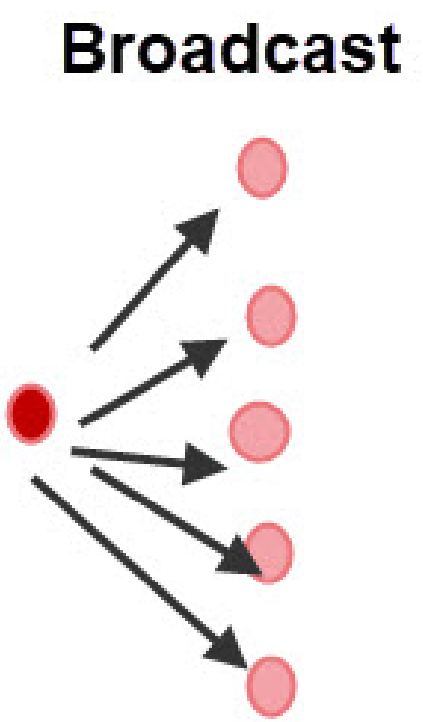
Practical, off-the-shelf mechanisms

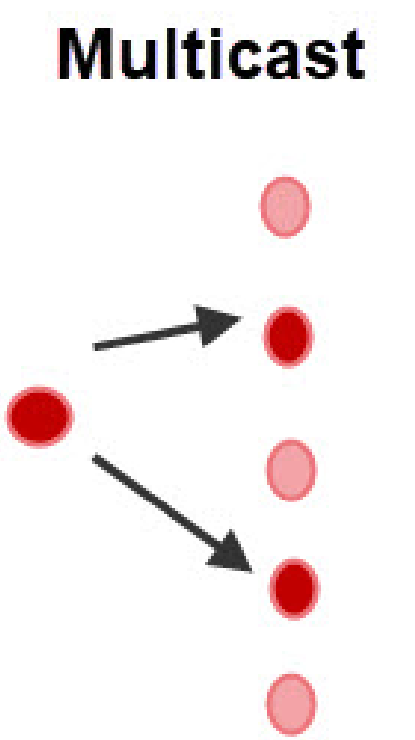


Send a message to all hosts on the local network segment (subnet)

No prior configuration and no group membership

Simple but not scalable





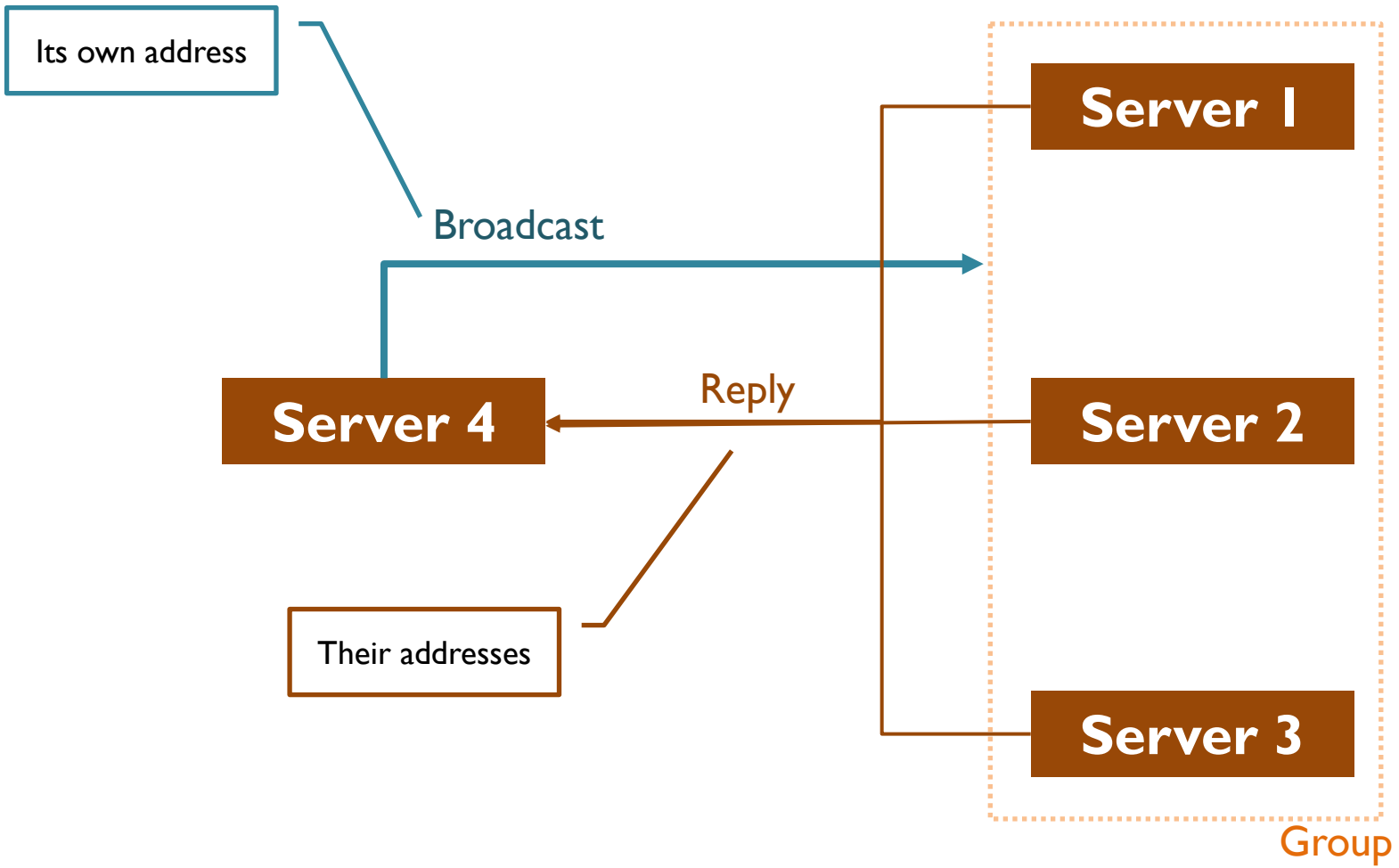
Send a message to all hosts that explicitly subscribed to a specific multicast group

Multicast group address must be known in advance

More scalable but requires group configuration

Problem 2

How to identify participants of a group and become a participant?



Possible procedure for broadcast-based discovery

- Each group participant maintains its own list of current group participants, called the *group view*

1. New group participant
   - Has no knowledge of any specific existing group participant
   - Knows only a *broadcast address* on the local network
   - Sends a broadcast announcement
     - Announcement includes the network address (IP:port) of the new participant
2. Each existing group participant
   - Continuously listens for broadcast announcements
   - Receives a broadcast announcement
   - Adds the new participant to its group view
   - Responds with a reply message
     - Reply message includes participant's own network address
3. New group participant
   - Collects all reply messages
   - Creates its own group view from the replies

Additional logic needed for ensuring consistency of the group view!

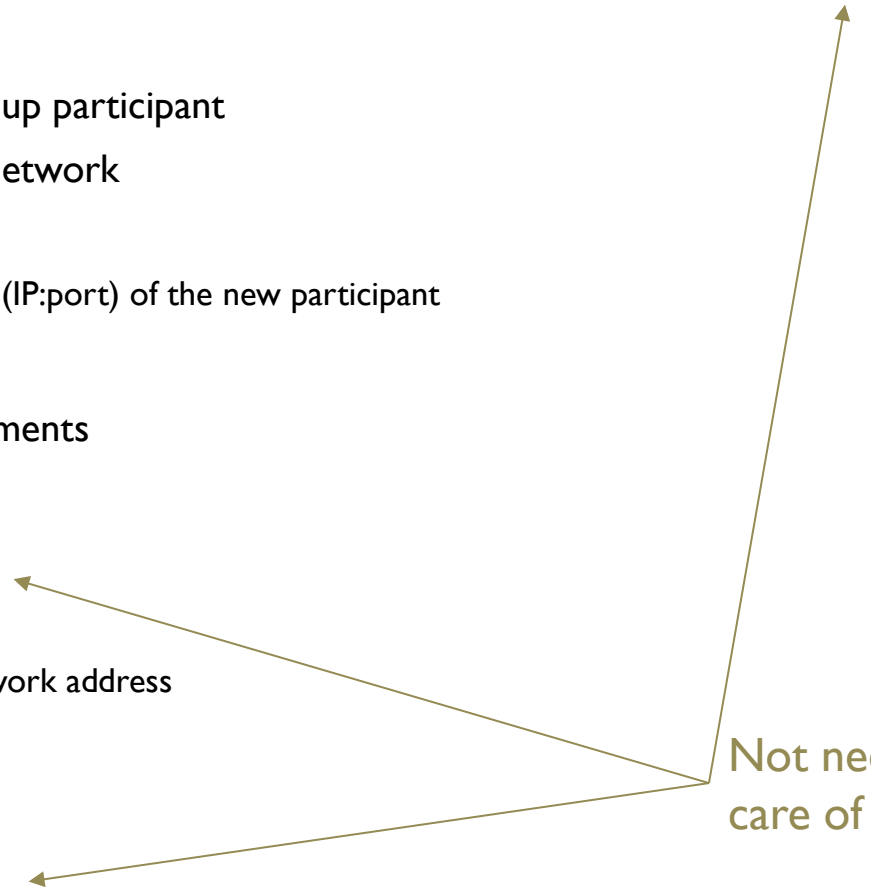
Alternatives to individual group views

Approach	Group view maintenance	Scalability	Fault tolerance	Overhead	Complexity
Distributed hash tables	Distributed	High	High	Moderate	Moderate
Gossip protocols	Distributed	High	High	Low	Moderate
Multicast groups	Implicit (inferred from multicast reception)	High	Moderate	Low	Low
Leader election	Centralised leader-maintained view	Moderate	Moderate	Moderate	Moderate

Possible procedure for broadcast-based discovery

- Each group participant maintains its own list of current group participants, called the *group view*

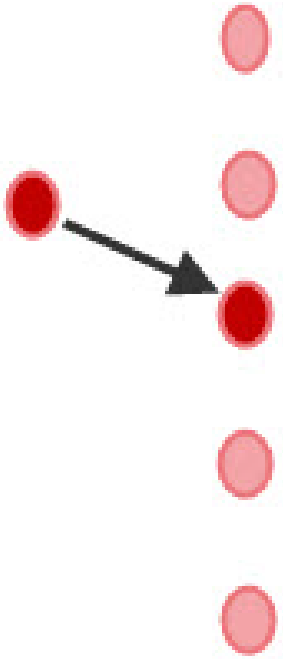
1. New group participant
   - Has no knowledge of any specific existing group participant
   - Knows only a *broadcast address* on the local network
   - Sends a broadcast announcement
     - Announcement includes the network address (IP:port) of the new participant
2. Each existing group participant
   - Continuously listens for broadcast announcements
   - Receives a broadcast announcement
   - Adds the new participant to its group view
   - Responds with a reply message
     - Reply message includes participant's own network address
3. New group participant
   - Collects all reply messages
   - Creates its own group view from the replies



Not needed if a leader takes care of the group view

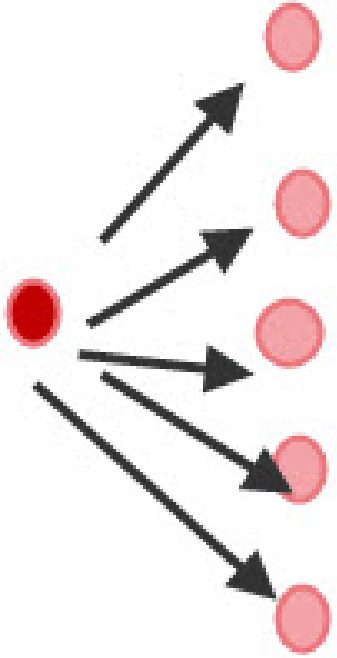
Additional logic needed for ensuring consistency of the group view!

Unicast



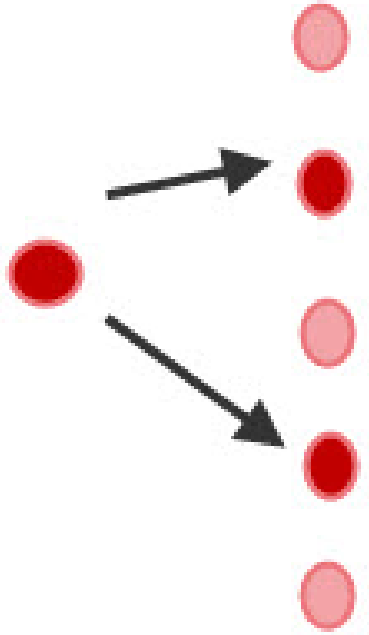
Class A,B,C
Transport TCP
and UDP

Broadcast



Hosts all 1's
Transport
UDP only

Multicast



Class D
Transport
UDP only

UDP broadcast

- UDP broadcast
  - Sends a UDP message to all hosts in a LAN
  - Stays within the local subnet, routers do not forward the message
- Broadcast address
  - Highest address in the local subnetwork (the address with all host bits set to 1 within the subnet)
  - Determined by the IP address and subnet mask of the device hosting the program
  - Some networks also support the limited broadcast address 255.255.255.255
  - Resources
    - Tutorial
    - Online calculator

Socket options

- Set socket options
  - `socket.setsockopt(level, optname, val)`
  - `SOL_SOCKET` for `level`
  - Some options behave differently across OSes (check documentation)
- Relevant options for `optname` for broadcast
  - `SO_BROADCAST` – allows sending to broadcast addresses
  - `SO_REUSEADDR` – allows rebinding to the same address/port (OS-dependent behaviour)
  - `SO_REUSEPORT` – allows multiple processes to bind the same port (not on Windows)
- Documentation
  - Linux
  - Windows
  - Mac OS

Summary of socket options

Option	Purpose	Use case
`SO_BROADCAST`	Enables a socket to send datagrams to a broadcast address	Required when sending UDP packets to broadcast addresses.
`SO_REUSEADDR`	Allows reuse of local addresses/ports, especially for rebinding after TIME_WAIT (OS-dependent behavior).	Useful for restarting servers quickly; on Unix-like systems can permit multiple UDP sockets to bind the same port.
`SO_REUSEPORT`	Allows multiple sockets/processes to bind the same port (Linux/macOS).	Used for multiple processes/threads to receive broadcast messages through the same port; enables kernel load balancing (Linux).

Option	Linux/Ubuntu	macOS	Windows
`SO_BROADCAST`	Supported with consistent semantics.	Supported with consistent semantics.	Supported with consistent semantics.
`SO_REUSEADDR`	Supported; permits multiple UDP sockets to bind the same port when configured appropriately.	Supported; behaves like Linux.	Supported, but semantics differ from Unix systems; generally does not allow multiple active binds to the same (IP, port) combination.
`SO_REUSEPORT`	Supported; enables multiple sockets to share a port with kernel load balancing.	Supported (≥10.9); allows multiple sockets on the same port.	Not supported.

UDP broadcast in Python

DYNAMIC DISCOVERY

Broadcast sender

- Create a socket
- Enable broadcast on the socket
- Send a broadcast announcement
- Wait for replies
- Receive a reply message
- Collect all reply messages
- Create a group view based on the replies

Broadcast listener

- Create a socket
- Enable the socket to share the port
- Bind the socket to the broadcast port
- Start listening for broadcast announcements
- Receive a broadcast announcement
- Update the local group view
- Send a reply message directly to the sender
- Continue listening for future broadcast announcements

Broadcast sender

```
def broadcast(ip, port, broadcast_announcement):
    # Create a UDP socket
    broadcast_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    # Enable permission to send to broadcast addresses
    broadcast_socket.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
    # Send the broadcast announcement
    broadcast_socket.sendto(str.encode(broadcast_announcement), (ip, port))

if __name__ == '__main__':
    # Broadcast address and port (example for a /24 network)
    BROADCAST_IP = "192.168.0.255"
    BROADCAST_PORT = 5973

    # Local host information
    my_host = socket.gethostname()
    my_ip = socket.gethostbyname(my_host)

    # Compose broadcast announcement
    announcement = my_ip + ' sent a broadcast message’
    # Send the broadcast announcement
    broadcast(BROADCAST_IP, BROADCAST_PORT, announcement)
```

Broadcast listener

```
# Listening port
BROADCAST_PORT = 5972

# Create a UDP socket
listen_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Allow multiple listeners (Linux/macOS semantics)
listen_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Bind to all local interfaces to receive broadcast packets
listen_socket.bind(“0.0.0.0", BROADCAST_PORT))

print("Listening to broadcast announcements")

while True:
    data, addr = listen_socket.recvfrom(1024)
    if data:
        print("Received a broadcast announcement:", data.decode() )
```

Considerations

Problem 2 (System availability objective)

- Each participant must actively discover others and be discoverable
- So, each participant runs both
  - A broadcast sender (advertises itself)
  - A broadcast listener (responds to new participants)
- Broadcast sender and broadcast listener run in separate processes/threads to avoid blocking

Considerations

Group view maintenance

- Each participant independently builds and maintains its own group view
  - Group views are local
  - May diverge → inconsistent system state
- Without coordination and consistency logic (e.g., ordering, deduplication, timeouts), group views will drift over time

Considerations

Problem 1 (Reachability objective)

- The proposed procedure is too costly and too general for this (no need to discover all participants)
- A new node acts only as a broadcast sender
- One or more broadcast listeners exist, typically one responds
- No group view is needed
- Simpler and lower traffic than for *System availability*

Considerations

Broadcast-based discovery

- Broadcast is limited to the local subnet (not routed across networks)
- Message volume grows with the number of participants
  - Broadcast can cause congestion, especially for large systems
- Multicast reduces traffic and is more scalable
  - But requires explicit group management and router support
- For large or multi-network systems, alternative discovery mechanisms are needed

Considerations

Troubleshooting

- If broadcast does not work
  - Ensure the program does not bind to a broadcast address
  - Ensure devices are on the same subnet
  - Use a router or other network hub to which you have control of to establish network
  - Check router/switch/firewall settings (broadcast may be blocked)
  - Home routers often block broadcast across Wi-Fi and Ethernet
- Incorrect use of `SO_REUSEADDR` may lead to an "Address already in use" error
  - One reason could be that `SO_REUSEADDR` is set after making the bind call → Set all socket options before calling bind()
  - Note: Windows semantics differ from Linux/macOS